



Modellierung und UML

1. Einführung

Prof. Dr. Michaela Huhn und Prof. Dr. Sebastian Bindick

Rechtlicher Hinweis und Copyright

Umgang mit dem Kursmaterial

- Das Kursmaterial ist durch das Copyright der Dozenten und/oder anderer Autoren geschützt.
- Das Material darf nur von Studierenden der angegebenen Fachhochschule und nur zu Ausbildungszwecken im Rahmen des angegebenen Kurses verwendet werden.
- Die Veröffentlichung oder Verbreitung des Materials ist strafbar und ausdrücklich untersagt. Dazu zählen das Veröffentlichen auf Webseiten, Onlinespeichern wie Dropbox, Verbreiten per Email, die Verwendung in Vorträgen oder Publikationen etc.

Modellierung und UML

- PO2018: Modellierung mit UML und BPMN
- Schwerpunkt: UML
 - BPMN nur am Rande
- Inhalte basieren auf der VL „Modellbasierte Softwareentwicklung“ von Prof. B. Rumpe (RWTH Aachen)
- Folienskript: B. Rumpe (RWTH Aachen), H. Grönniger (jetzt: Hochschule Hannover)
 - Erweiterungen von M. Huhn und S. Bindick

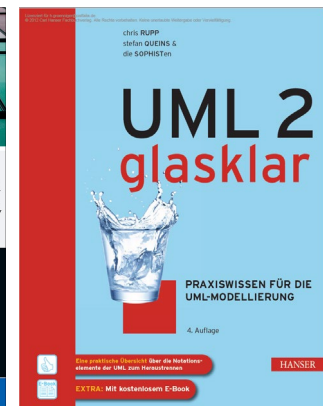
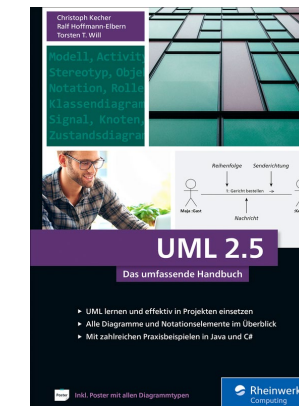
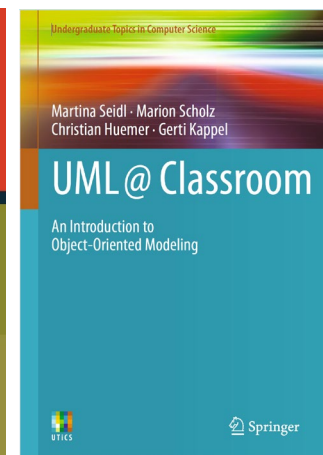
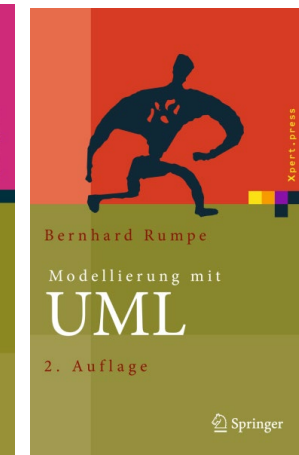
Ankündigungen, Material etc. in Moodle

- <https://moodle.ostfalia.de/course/view.php?id=15455>

Organisatorisches

Literatur

- B. Rumpe: (Agile) Modellierung mit der UML, Springer 2011 bzw. 2012 (zwei Bücher)
- C. Rupp, S. Queins. UML 2 glasklar. Hanser 2012
- M. Seidl, M. Scholz, Chr. Huemer, G. Kappel. UML@Classroom. Springer 2012
- OMG (Object Management Group) - UML Specification
<https://www.omg.org/spec/UML/2.5.1/PDF>
- Chr. Kecher, A. Salvanos, R. Hoffmann-Elbern. UML 2.5, 7. Auflage, Rheinwerk 2021



Organisatorisches

- Vorlesung
 - Mitarbeit, Fragen, Verbesserungsvorschläge erwünscht
- Übung
 - Vorstellung von Übungsaufgaben, hier ist umso mehr aktive Mitarbeit gewünscht
- Modulprüfung
 - 70% Klausur, geplant mit [Artemis](#)
 - 30% Projekt
 - 10% Bonuspunkte
 - Artemis-Aufgaben (automatisiert bewertet): Anteil der gelösten und anrechenbaren Aufgabenpunkte
 - Es müssen 50% der Klausurpunkte erreicht werden

Organisatorisches

Projekt

- Auswahl einer Modellierungsaufgabe in UML
 - Themenliste: [Projektvorschläge in Moodle](#)
 - Inhalte
 - Motivation, Systembeschreibung, Verfeinerung der Funktionen und Use Cases
 - Sicht auf die Struktur (wichtige Aspekte, mindestens Klassendiagramme + OCL)
 - Sicht auf das Verhalten (wichtige Aspekte, mindestens Aktivitätsdiagramm UND Statechart)
- Teamgröße: 4
- Vorgehen:
 - Themenwahl (max. 3 Teams für ein Thema): 11.03.2026 um 15:00 über Moodle-Aktivität
 - Abgabe (Projektausschnitt), Use Cases + Beschreibung: 22.03.2026 (2 Punkte)
 - Abgabe Strukturmodelle 09.04.2026 (9 Punkte)
 - Abgabe Verhaltensmodelle 13.05.2026 (9 Punkte)
 - Vorstellung der Modellierung als Ganzes ab 27.05.26 (10 Punkte –Qualität, Dokumentation, Vortrag)

Projekt Modellierung

- Richtwerte
 - Aufwand: 30-40h pro Teammitglied
 - D.h. 60-75 min Aufwand pro Person und Punkt
- Bewertungskriterien u.a.
 - Notation syntaktisch und semantisch korrekt angewendet
 - Zweck der Modellierung im Software-Design der Kern-Anwendung erkennbar
 - „anspruchsvolle Elemente“ wie OCL sinnvoll eingesetzt
 - widerspruchsfrei + konsistent
 - angemessene, verständliche Abstraktion für die gewählte Aufgaben

Inhalt

0.	Organisatorisches
1.	Begriffserklärung und Ziele & die UML
2	Die erste Diagrammart: Use Case Diagramme
3.	Strukturmodellierung und Klassendiagramme
4.	Objektdiagramme
5.	Aktivitätsdiagramme
6.	Statecharts

6.	Sequenzdiagramme
7.	OCL
8.	Weitere UML Diagrammarten
9.	

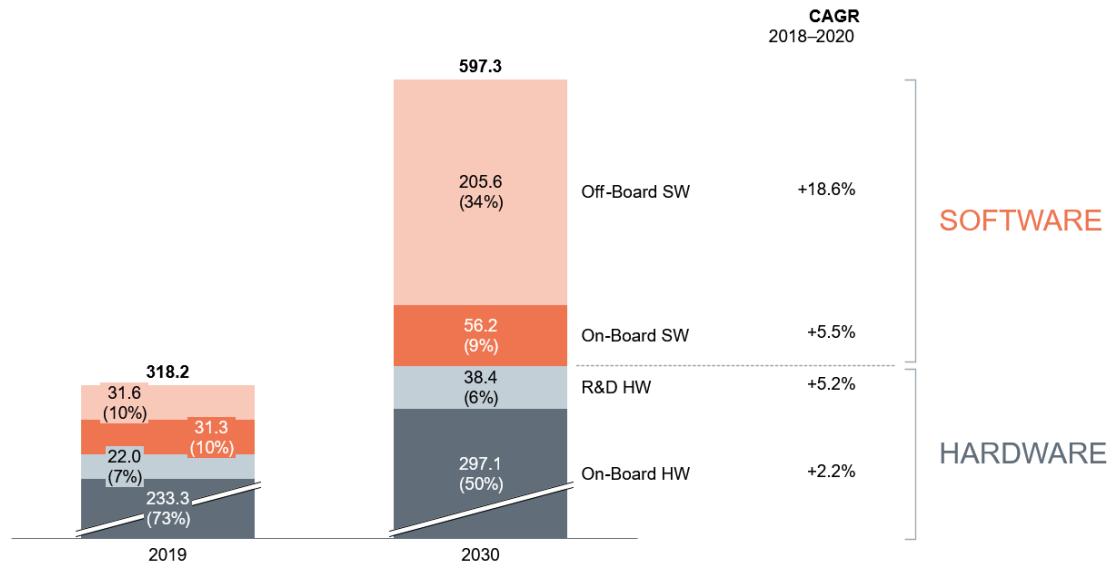
EINFÜHRUNG

Herausforderungen in der Softwareentwicklung

- Software-Anteil an Produkten nimmt weiterhin dramatisch zu
- Komplexitätssteigerungen um Größenordnungen
- Eingebettete Software und Integration von Anwendungen:
 - Kühlschrank, Smart Phone, Auto, Flugzeug
- Fachliche Probleme scheiternder Projekte:
 - Software wird zu spät fertig
 - Endnutzer nicht einbezogen oder unklare Anforderungen
 - Software ist schlecht dokumentiert/kommentiert und kann nicht weiter entwickelt werden
 - Quellcode fehlt
 - Technische Umgebung wechselt
 - Geschäftsmodell / Anforderungen wechseln

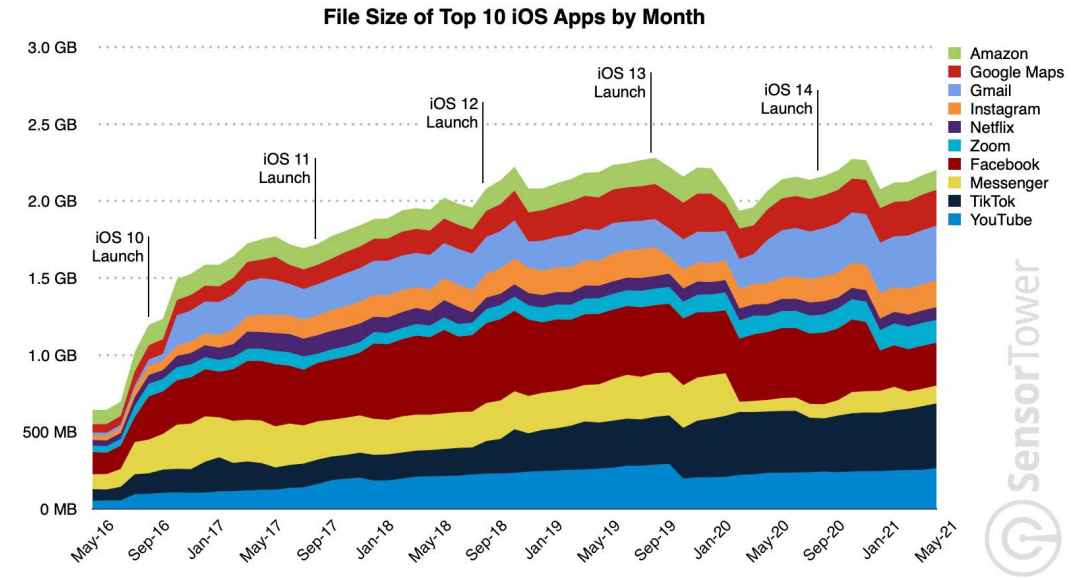
Trends in der Softwareentwicklung

Automobil-Branche



berylls

iOS Apps



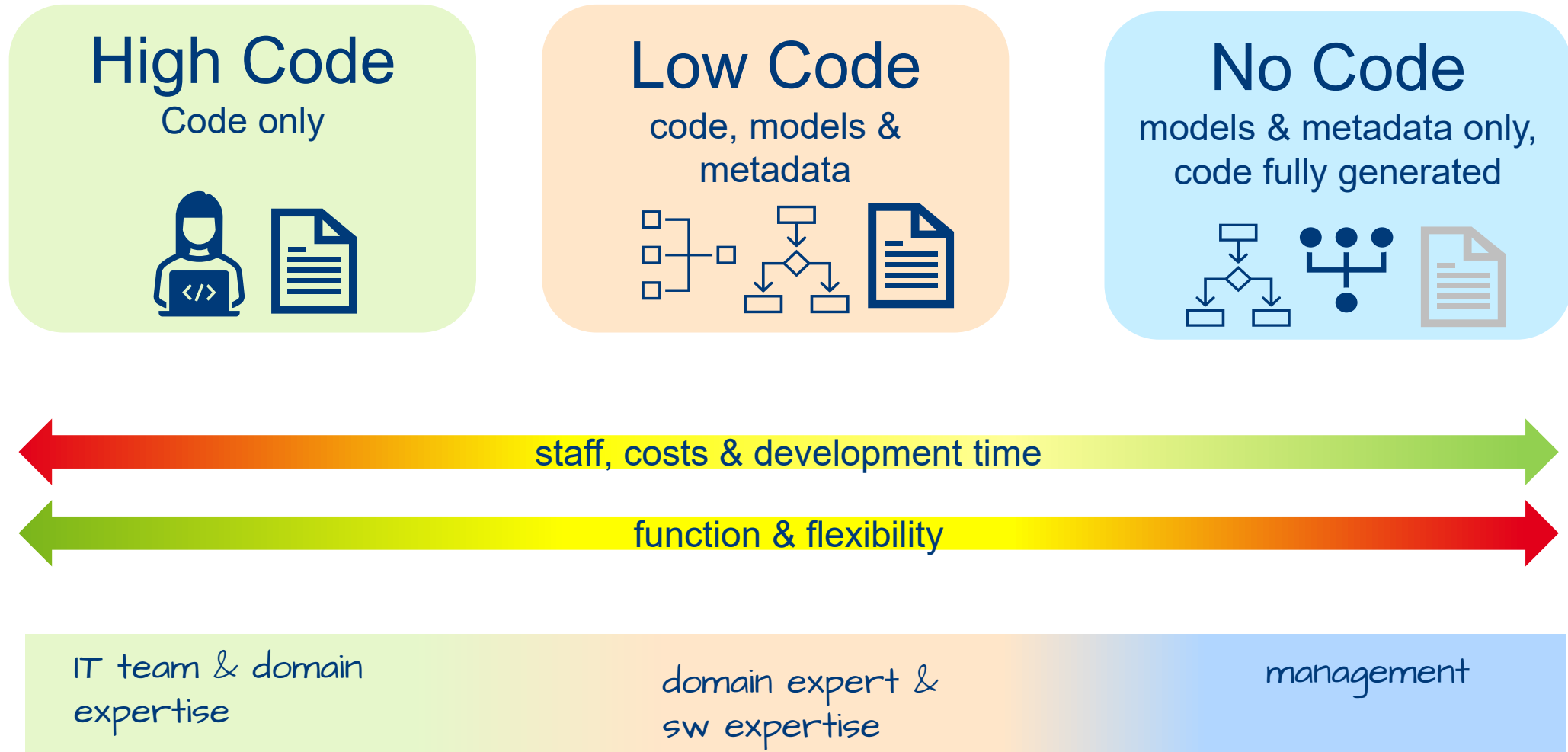
SensorTower

SensorTower Data That Drives App Growth

sensortower.com

- Es wird immer mehr Software entwickelt
- Die entwickelte Software wird immer komplexer

Low Code Trend



* Abbildung angelehnt an Quelle: STEPlus.co, 2023

Portfolio von Softwareentwicklungs-Techniken

Ziel: Vergrößerung des Portfolios von Techniken, Konzepten und Werkzeugen

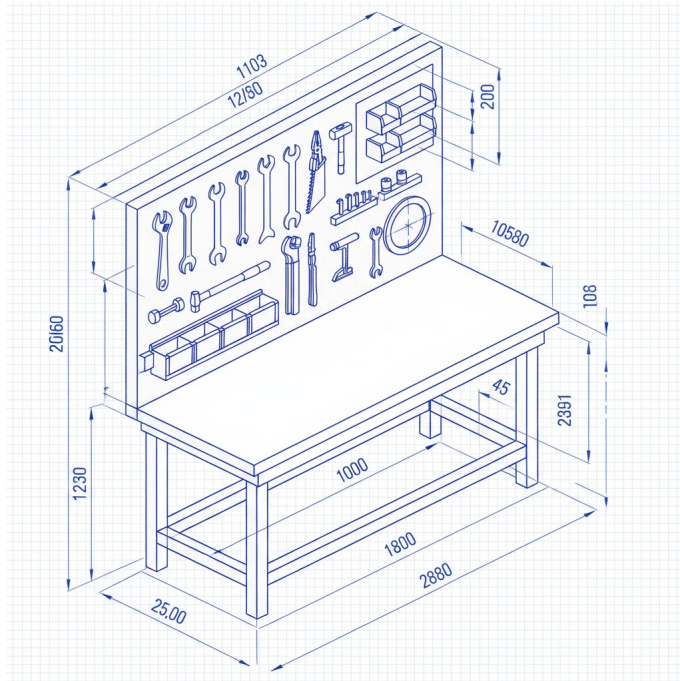


Bild generiert mit ChatGPT

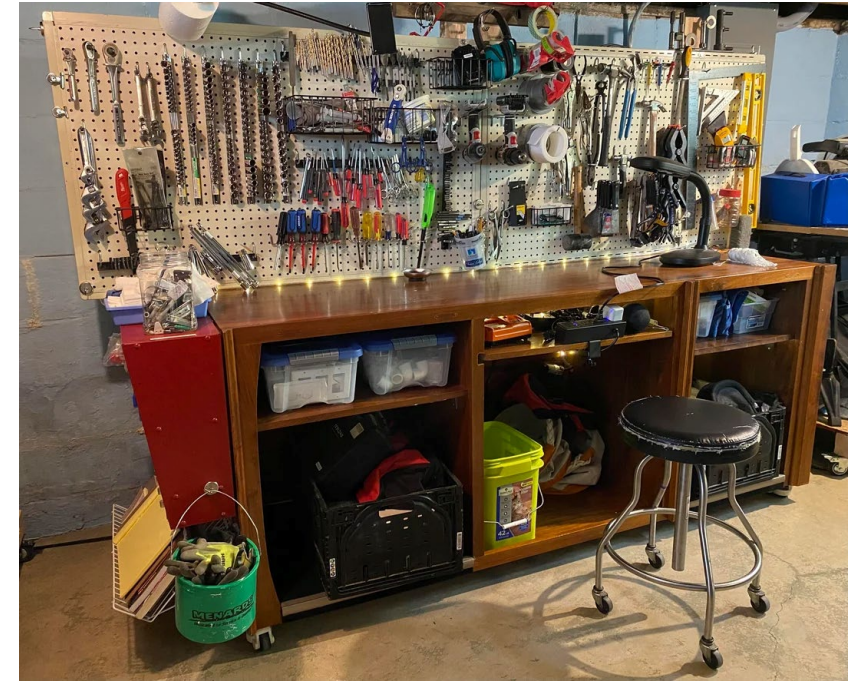
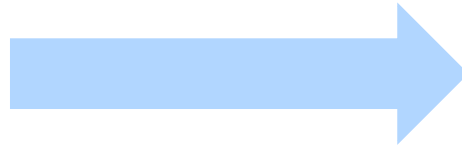


Foto: Larry W. Biddingen

so dass für jedes Problem ein angemessenes Verfahren von den Entwickler/-innen beherrscht wird.

Portfolio von Softwareentwicklungs-Techniken

Welche Techniken, Konzepte, Werkzeuge kennen Sie?

Portfolio von Softwareentwicklungs-Techniken

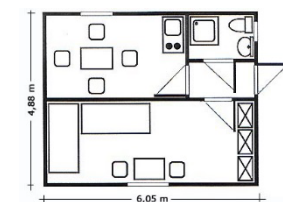
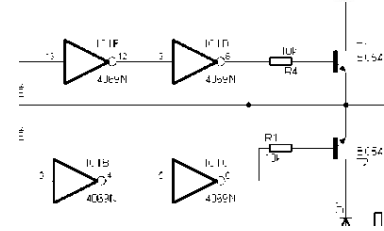
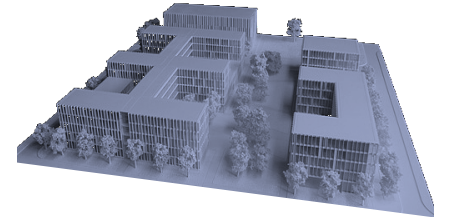
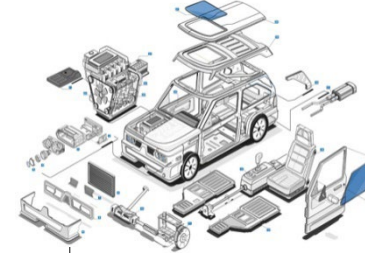
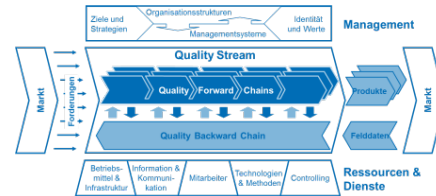
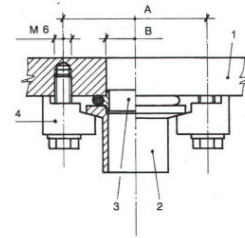
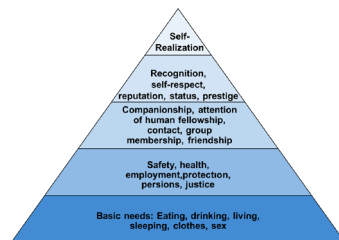
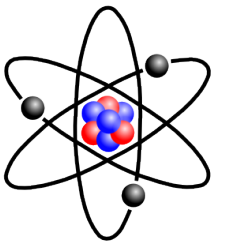
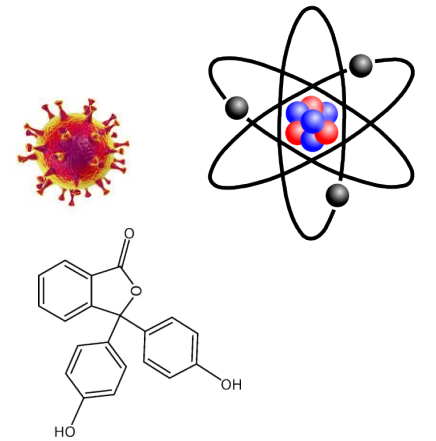
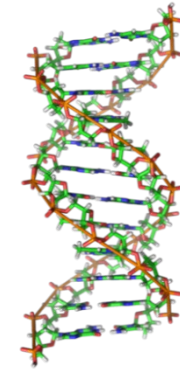
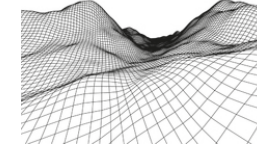
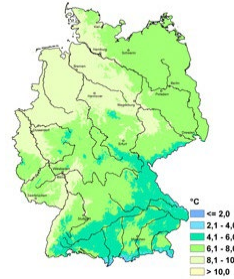
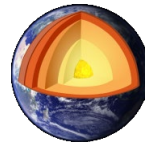
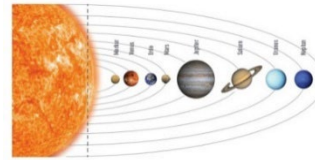
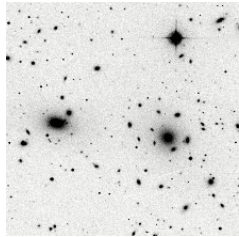
Beispiele:

- Konzepte: Hierarchische Zerlegung („Divide Et Impera“),
Modularisierung,
Zustandsbasierte Entwicklung
- Werkzeuge: Compiler, SVN / Git, Eclipse
- Methoden: CRC-Karten zur Anforderungserhebung,
Reviews, Testverfahren
- Sprachen: zur Dokumentation,
Implementierung,
Modellierung

Was ist ein Modell?

Beispiele:

Was ist ein Modell?



Der Modellbegriff

Ein Modell ist seinem Wesen nach eine in Maßstab, Detailliertheit und/oder Funktionalität verkürzte beziehungsweise abstrahierte Darstellung des originalen Systems. (Stachowiak 1973*)

Ein Modell ist eine vereinfachte, auf ein bestimmtes Ziel hin ausgerichtete Darstellung der Funktion eines Gegenstands oder des Ablaufs eines Sachverhalts, die eine Untersuchung oder eine Erforschung erleichtert oder erst möglich macht. (Balzert 2000**)

*Quelle: H. Stachowiak: Der Modellbegriff in der Erkenntnistheorie, Zeitschrift für allgemeine Wissenschaftstheorie, 1980

** H. Balzert, Lehrbuch der Softwaretechnik, 2000

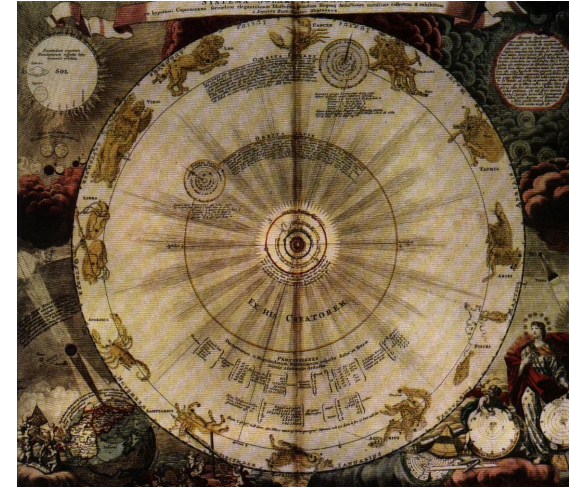
Modell – Konsequenzen

- 1) Zu dem Modell gibt es ein Original.
- 2) Das Modell zeigt nicht alles, es ist eine Vereinfachung, Abstraktion.
- 3) Modelle werden mit einem Ziel erstellt und verwendet, um Eigenschaften des Originals zu studieren.

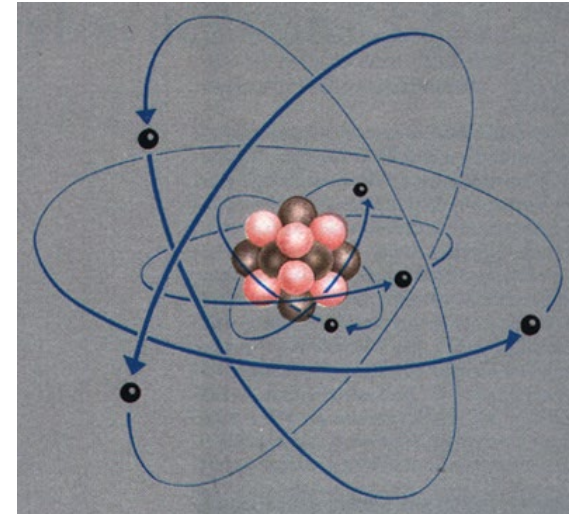
- Präskriptive Modelle: Zuerst das Modell, dann die Realisierung / das Original
- Deskriptive Modelle: das existierende Original wird modelliert

Einschränkungen

Nicht alle Modelle sind korrekt



Einige Modelle gelten nur in gewissen Grenzen



Wer / was ist kein Modell?



Was ist Modell, was Original?



(Rene Magritte)

Modellierung in der Softwaretechnik

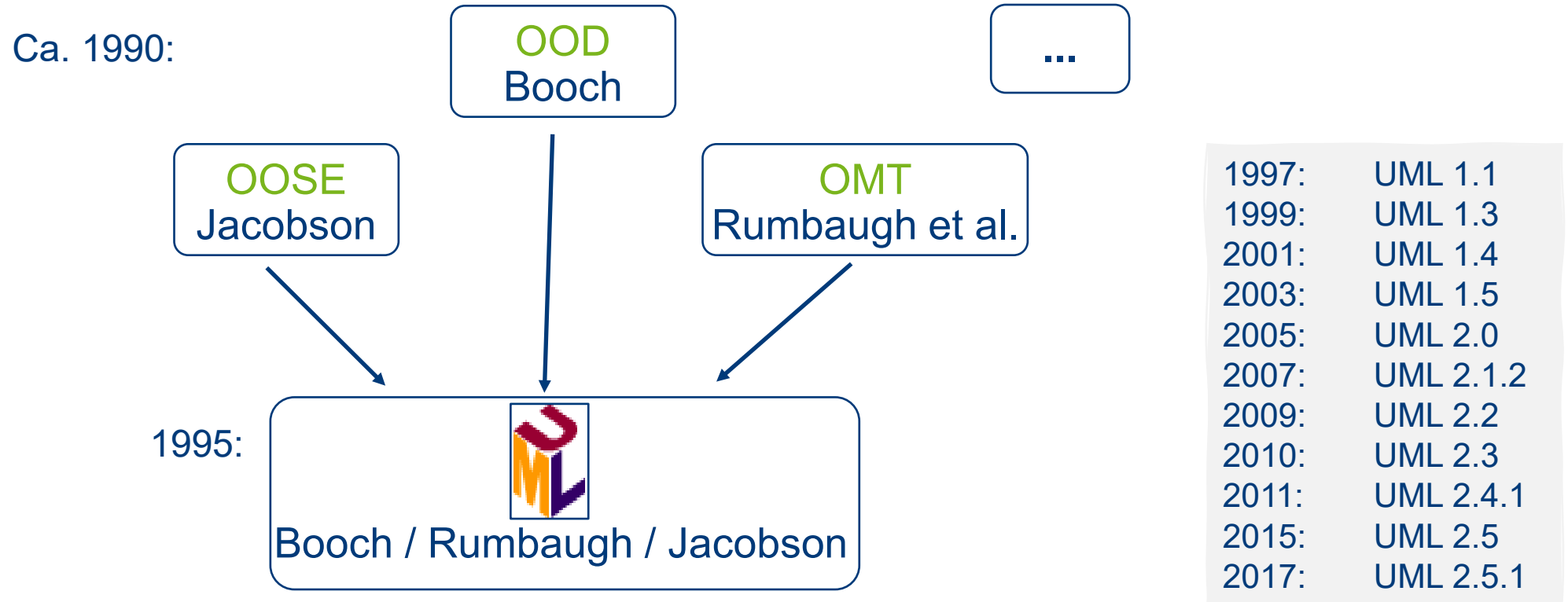
- Industriestandard: **Unified Modeling Language**
 - 13 Diagrammtechniken (Klassendiagramme, Statecharts etc.)

Aber auch:

- Petri Netze
- Datenflussdiagramme
- Entity/Relationship-Model
- Nassi-Schneidermann-Diagramme
- SDL
- Endliche Automaten
- Jackson Structured Diagrams
- Business Process Modeling Notation (BPMN)
- Algebraische Spezifikation
- Kontrollflussdiagramme
- Grammatiken
- Reguläre Ausdrücke
- ...

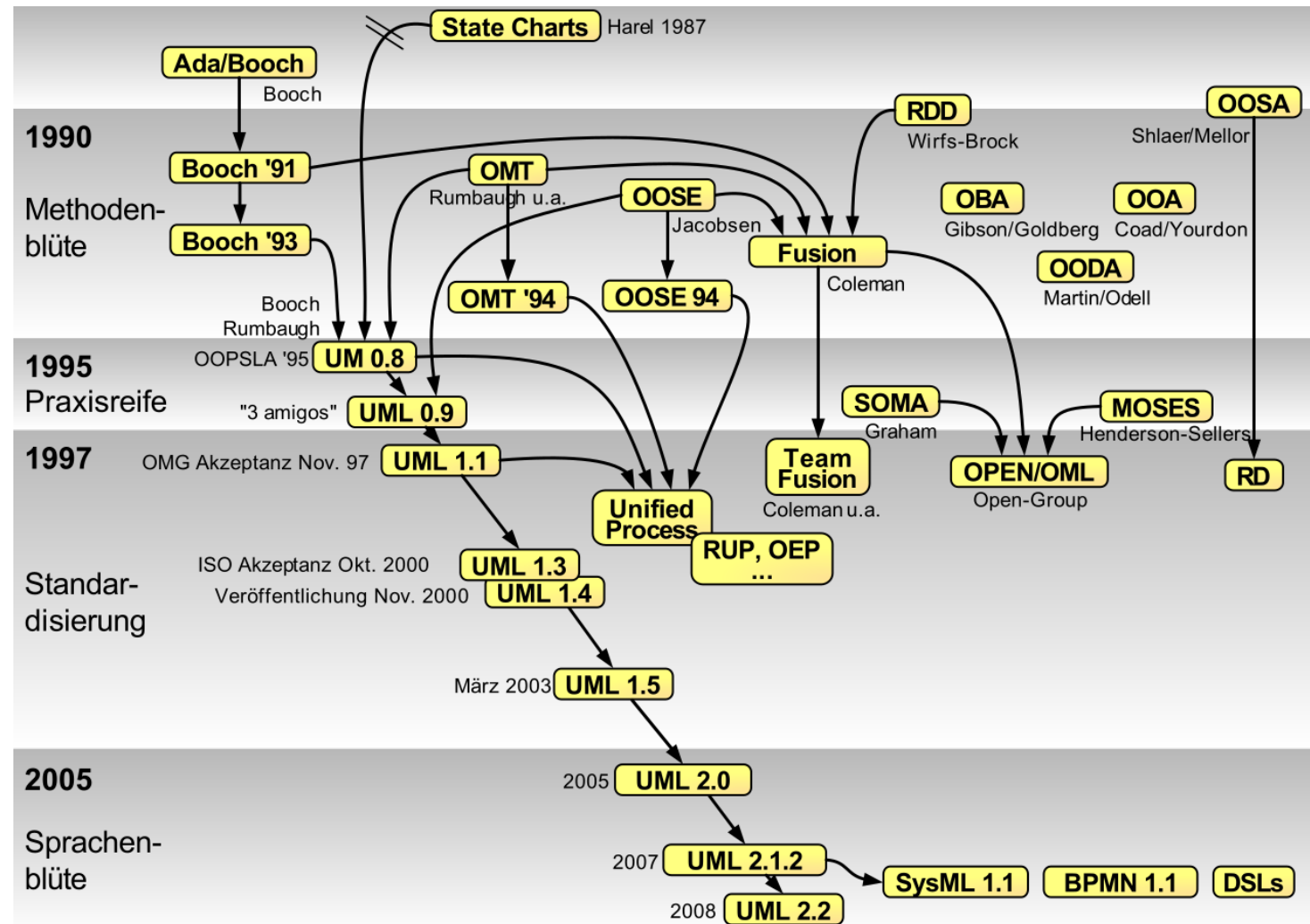
UML – ERSTER ÜBERBLICK

UML – Unified Modeling Language



UML ist eine Notation zur objektorientierten Modellierung

UML ist Industriestandard der OMG (Object Management Group)



- Mai 2010: Version 2.3 mit Fehlerkorrekturen am Metamodell und Schärfungen der Semantik von Modellelementen im Spezifikationsdokument der UML
- Juni 2015: Version 2.5
- Dezember 2017: Version 2.5.1 – aktuelle Fassung

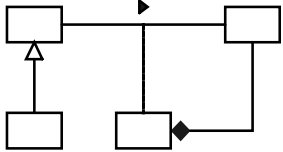
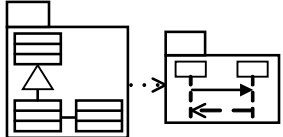
Unified Modeling Language

- Graphische Modellierungssprache für Software-Systeme
- Vereinigung mehrerer Vorgänger-Methoden
- Sprachmittel zur Spezifikation, Kommunikation und Dokumentation
 - zwischen Entwicklern
 - Entwicklern mit Anwendern
- Standardisiert seit September 1997 von der OMG
- Entwickelt von
Booch, Rumbaugh, Jacobson, Selic, Kobryn, Cook und vielen anderen ...
- Besteht aus:
 - Einer Menge von Modellierungskonzepten
 - Einer konkreten Notation

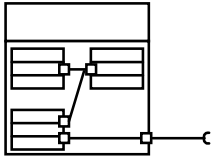
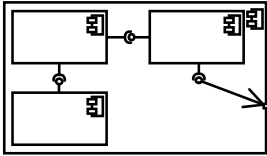
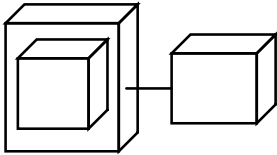
Ziele der UML

- Beschreibung **wesentlicher Eigenschaften** des Systems wie in einem Bauplan
- **Strukturierung des Problems und der Lösung**
- **Abstraktion** von Implementierungsdetails
- Definition verschiedener **Sichten**:
 - Software/System-Architektur
 - Interaktion zwischen Komponenten
 - Verhalten von Komponenten
 - Implementierung
 - Physische Verteilung
 - Aufgabenverteilung und Workflows
- Nutzbar in allen Phasen des Software Entwicklungszyklus

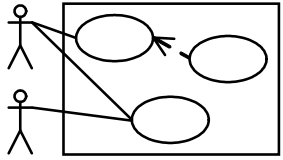
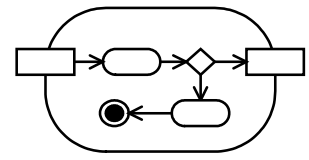
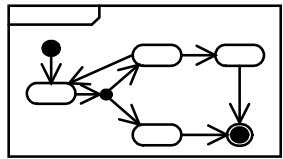
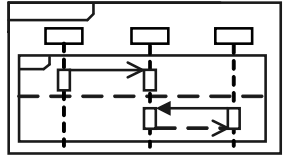
UML Diagrammtypen (aus UML 2 glasklar)

Diagrammtyp	Diese zentrale Frage beantwortet das Diagramm	Stärken
Klassendiagramm  <pre> classDiagram class A class B class C class D class E A -- > B A -- > C B --> D C --> D D --> E </pre>	Aus welchen Klassen besteht mein System und wie stehen diese untereinander in Beziehung?	Beschreibt die statische Struktur des Systems. Enthält alle relevanten Strukturzusammenhänge/Datentypen. Brücke zu dynamischen Diagrammen. Normalerweise unverzichtbar.
Paketdiagramm  <pre> packageDiagram package P1 package P2 package P3 P1 --> P2 P2 --> P3 </pre>	Wie kann ich mein Modell so schneiden, dass ich den Überblick bewahre?	Logische Zusammenfassung von Modellelementen. Modellierung von Abhängigkeiten/ Inklusion möglich.
Objektdiagramm  <pre> classDiagram class A class B class C class D A -- > B A -- > C B --> D C --> D </pre>	Welche innere Struktur besitzt mein System zu einem bestimmten Zeitpunkt zur Laufzeit (Klassendiagramm-schnappschuss)?	Zeigt Objekte u. Attributbelegungen zu einem bestimmten Zeitpunkt. Verwendung beispielhaft zur Veranschaulichung Detailniveau wie im Klassen-diagramm. Sehr gute Darstellung von Mengenverhältnissen.

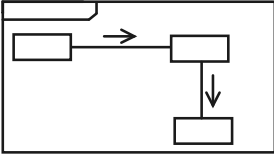
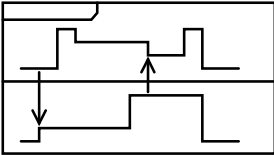
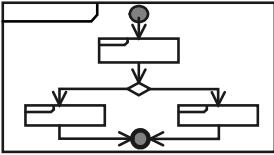
UML Diagrammtypen

Diagrammtyp	Diese zentrale Frage beantwortet das Diagramm	Stärken
Kompositionsstrukturdiagramm 	Wie sieht das Innenleben einer Klasse, einer Komponente, eines Systemteils aus?	Ideal für die Top-Down-Modellierung des Systems (Ganz-Teil-Hierarchien). Zeigt Teile eines „Gesamtelements“ und deren Mengenverhältnisse. Präzise Modellierung der Teile-Beziehungen über spezielle Schnittstellen (Ports) möglich.
Komponentendiagramm 	Wie werden meine Klassen zu wieder verwendbaren, verwaltbaren Komponenten zusammengefasst und wie stehen diese in Beziehung?	Zeigt Organisation und Abhängigkeiten einzelner technischer Systemkomponenten. Modellierung angebotener und benötigter Schnittstellen möglich.
Verteilungsdiagramm 	Wie sieht das Einsatzumfeld (Hardware, Server, Datenbanken, ...) des Systems aus? Wie werden die Komponenten zur Laufzeit wohin verteilt?	Zeigt das Laufzeitumfeld des Systems mit den „greifbaren“ Systemteilen. Darstellung von „Softwareservern“ möglich. Hohes Abstraktionsniveau, kaum Notationselemente.

UML Diagrammtypen

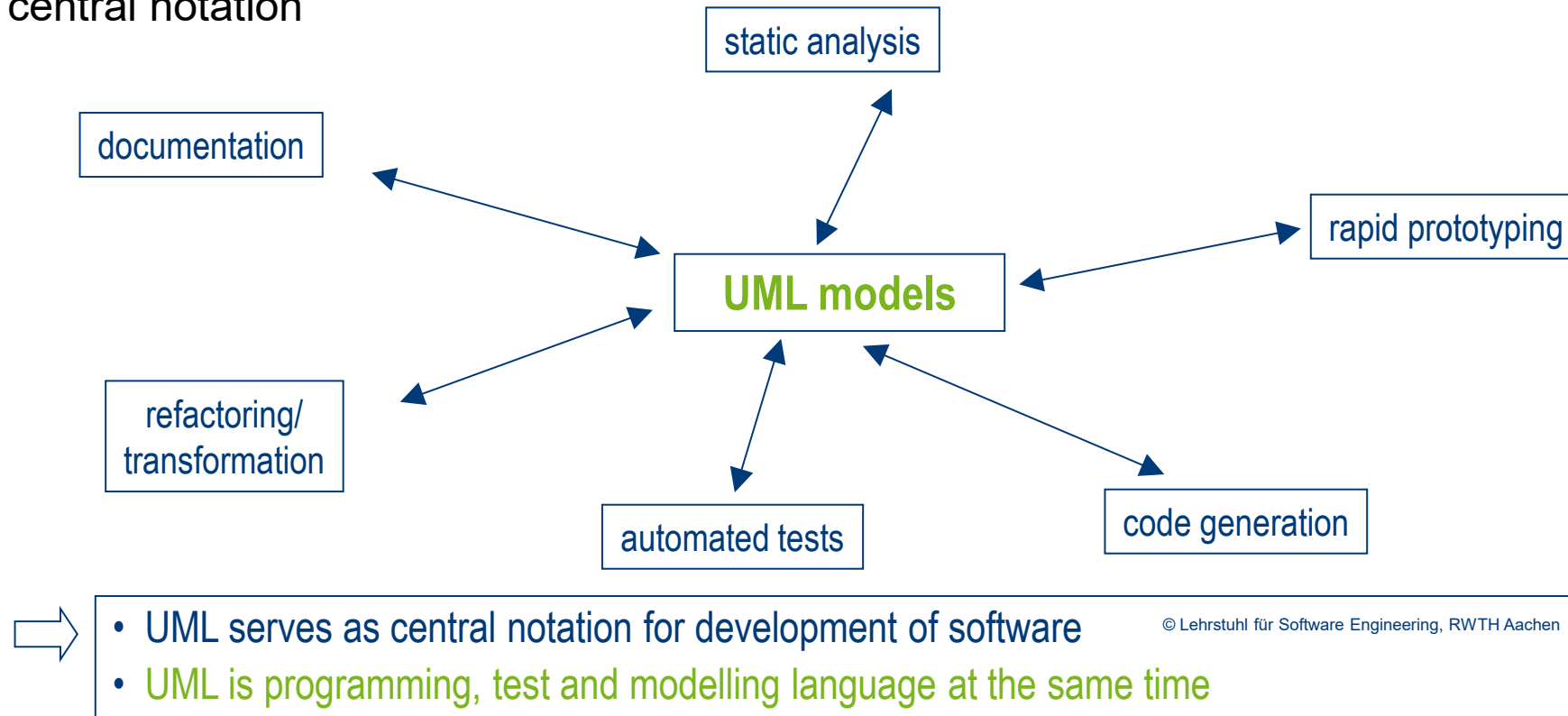
Diagrammtyp	Diese zentrale Frage beantwortet das Diagramm	Stärken
Use-Case-Diagramm 	Was leistet mein System für seine Umwelt (Nachbarsysteme, Stakeholder)?	Außensicht auf das System. Geeignet zur Kontextabgrenzung. Hohes Abstraktionsniveau, einfache Notationsmittel.
Aktivitätsdiagramm 	Wie läuft ein bestimmter fluss-orientierter Prozess oder ein Algorithmus ab?	Sehr detaillierte Visualisierung von Abläufen mit Bedingungen, Schleifen, Verzweigungen. Parallelisierung und Synchronisation. Darstellung von Datenflüssen.
Zustandsautomat 	Welche Zustände kann ein Objekt, eine Schnittstelle, ein Use Case, ... bei welchen Ereignissen annehmen?	Präzise Abbildung eines Zustandsmodells mit Zuständen, Ereignissen, Nebenläufigkeit, Bedingungen, Ein- und Austrittsaktionen. Schachtelung möglich.
Sequenzdiagramm 	Wer tauscht mit wem welche Informationen in welcher Reihenfolge aus?	Darstellung des Informationsaustauschs zwischen Kommunikationspartnern Sehr präzise Darstellung der zeitlichen Abfolge auch mit Nebenläufigkeit.

UML Diagrammtypen

Diagrammtyp	Diese zentrale Frage beantwortet das Diagramm	Stärken
Kommunikationsdiagramm 	Wer kommuniziert mit wem? Wer „arbeitet“ im System zusammen?	Stellt den Informationsaustausch zwischen Kommunikationspartnern dar. Überblick steht im Vordergrund (Details und zeitliche Abfolge weniger wichtig).
Timingdiagramm 	Wann befinden sich verschiedene Interaktionspartner in welchem Zustand?	Visualisiert das exakte zeitliche Verhalten von Klassen, Schnittstellen,.. Geeignet für die Detailbetrachtungen, bei denen es wichtig ist, dass ein Ereignis zum richtigen Zeitpunkt eintritt.
Interaktionsübersichtsdiagramm 	Wann läuft welche Interaktion ab?	Verbindet Interaktionsdiagramme (Sequenz-, Kommunikation- und Timingdiagramme) auf Top-Level-Ebene. Hohes Abstraktionsniveau.

Modellbasierte Entwicklung mit UML

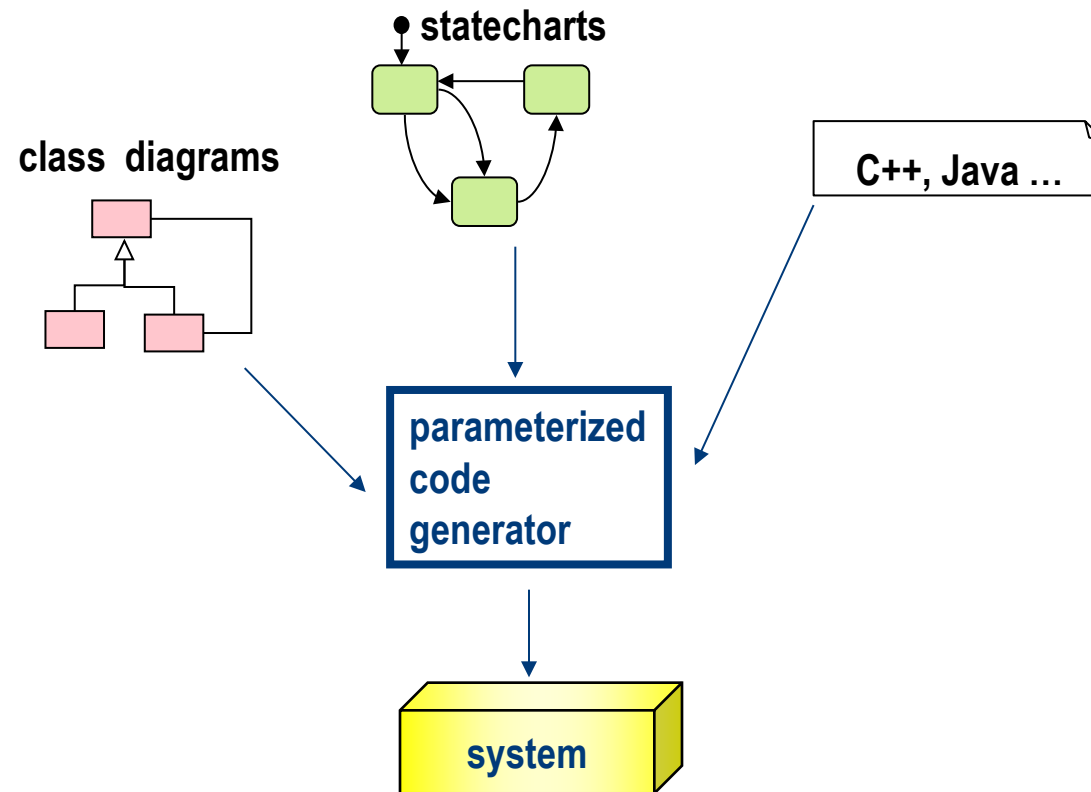
Models as a central notation



- betont den konstruktiven und generativen Einsatz von Modellen;
- oft wird UML als informelle Notation verwendet, insb. zur Kommunikation von Menschen über ein System

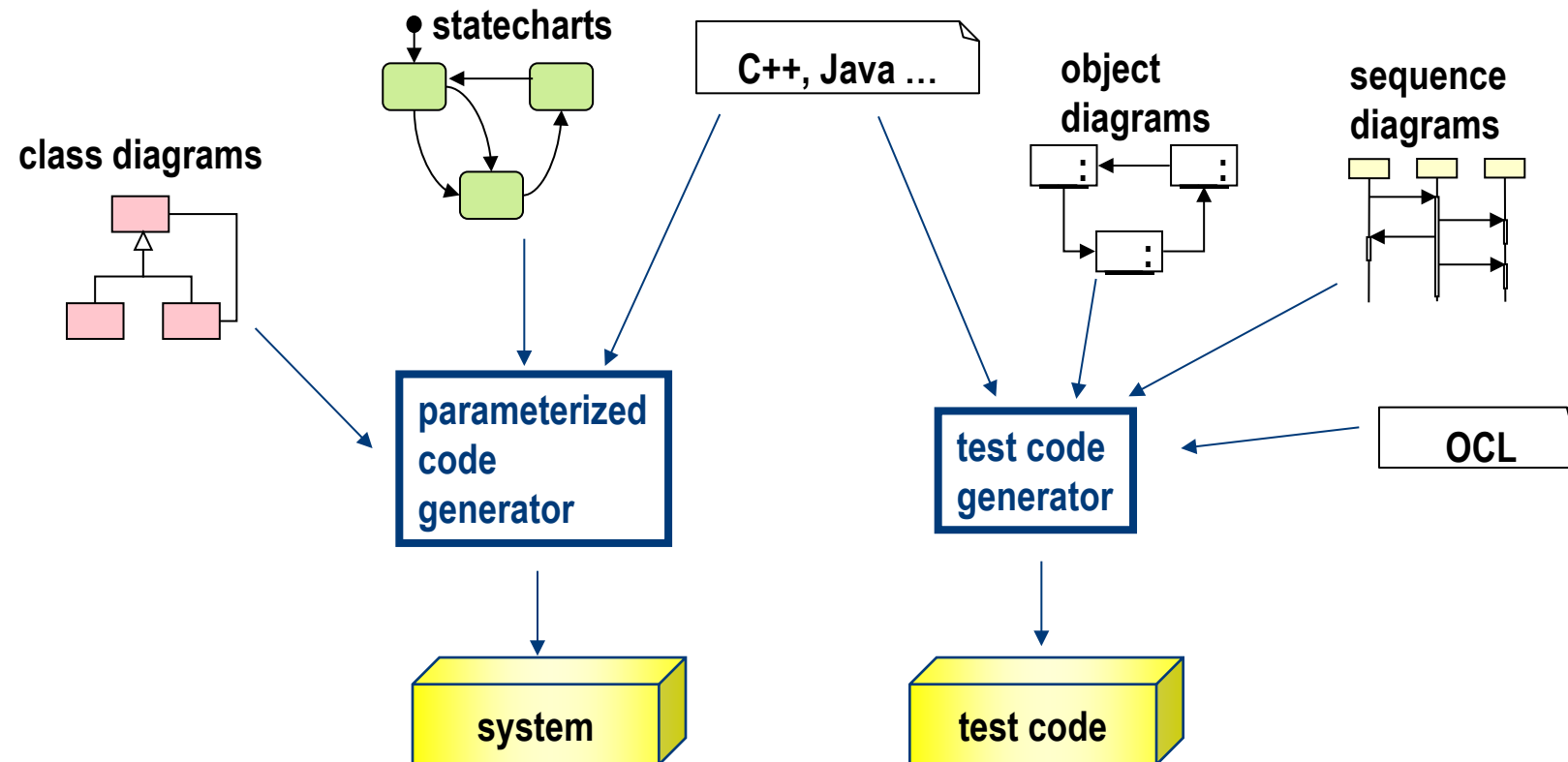
UML-basierte Modellierung

UML + Code-Rümpfe erlauben Code-Generierung



UML-basierte Modellierung

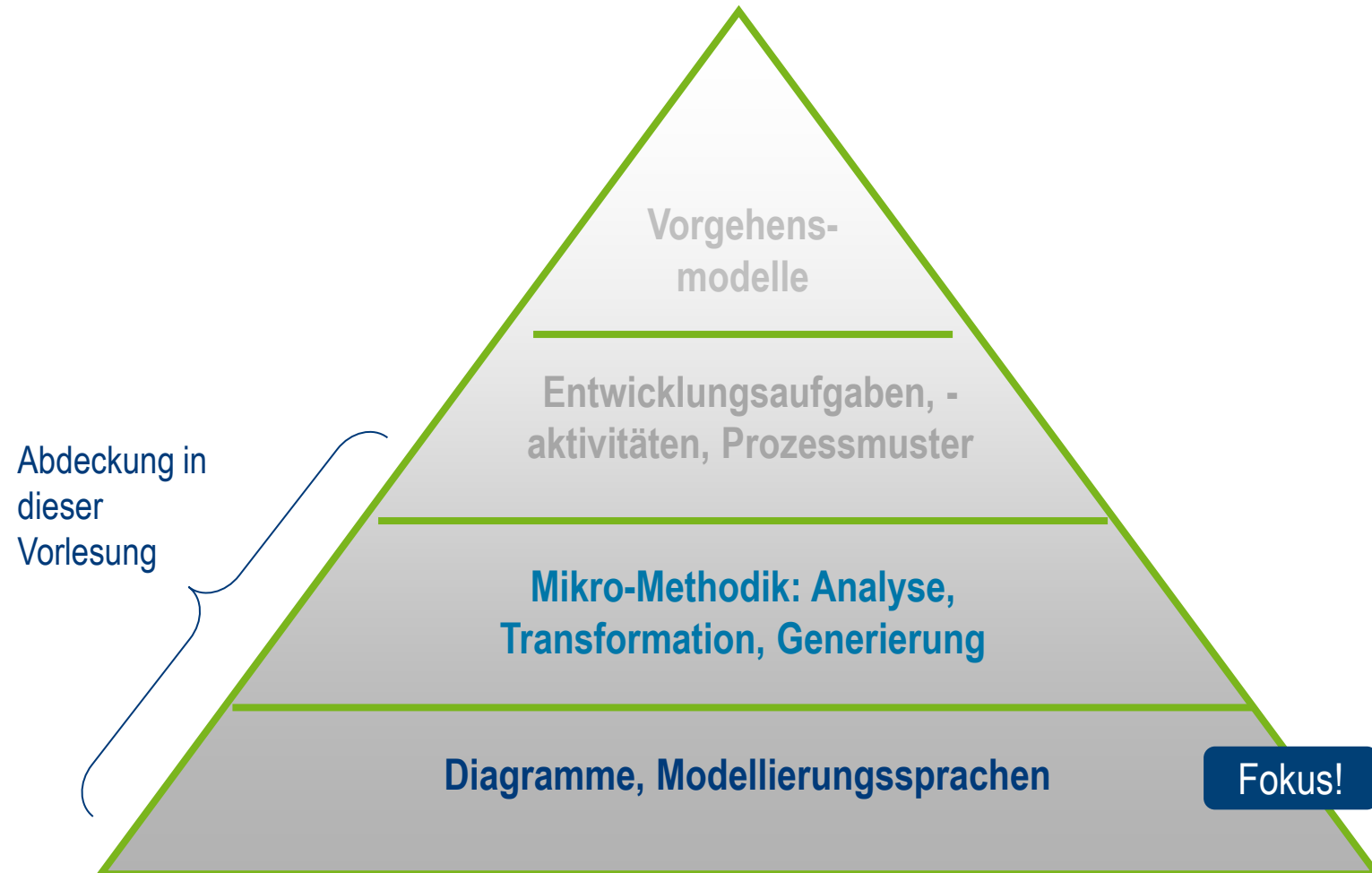
UML + Code-Rümpfe erlauben Code & Test-Modellierung



⇒ Code- und Testmodelle prüfen wechselseitig Korrektheit

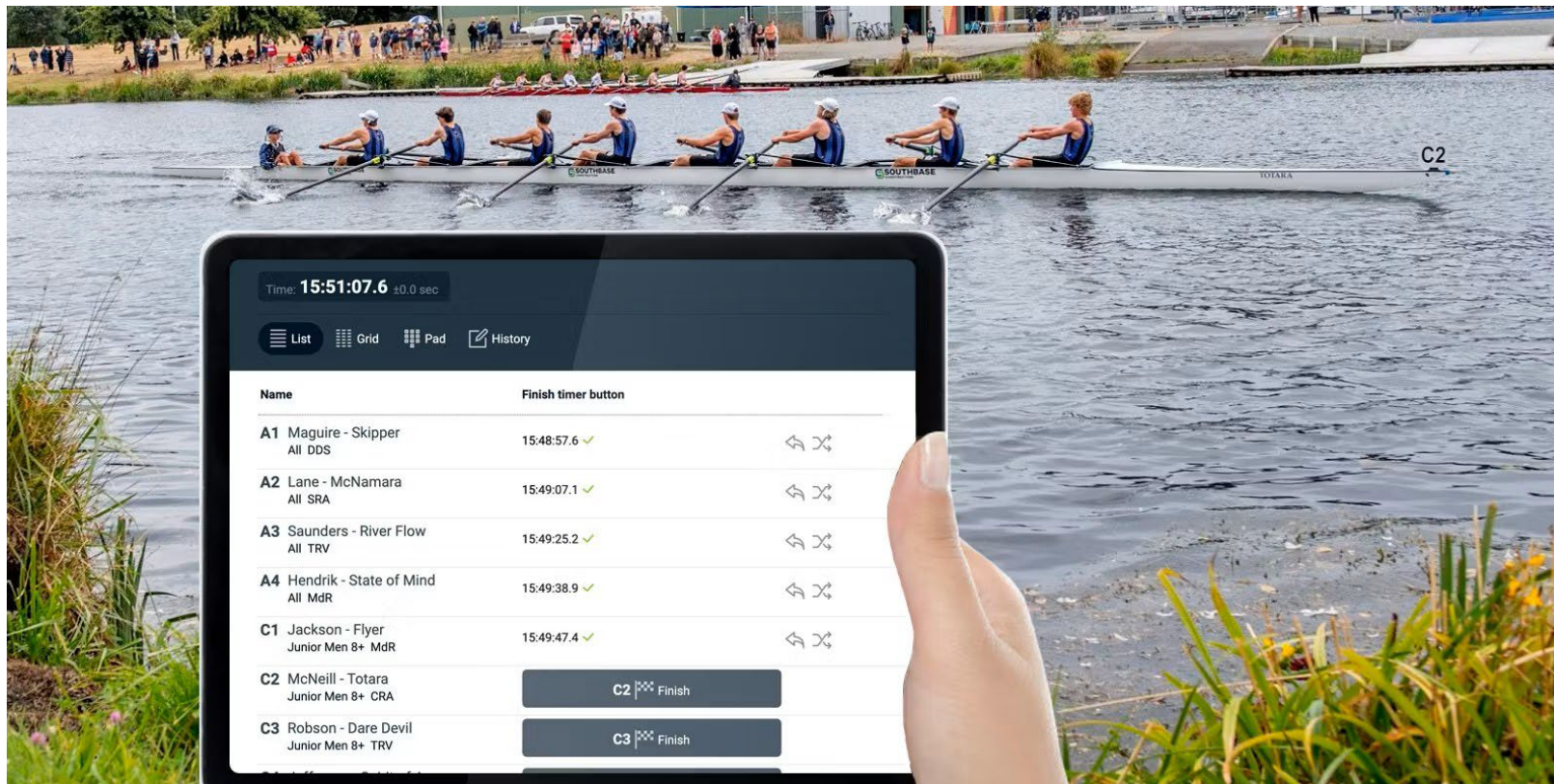
Grundlagen der Modellbildung

Methodik-Pyramide



Wiederkehrendes Beispiel: Zeitmessung für Amateur-Sportwettrennen

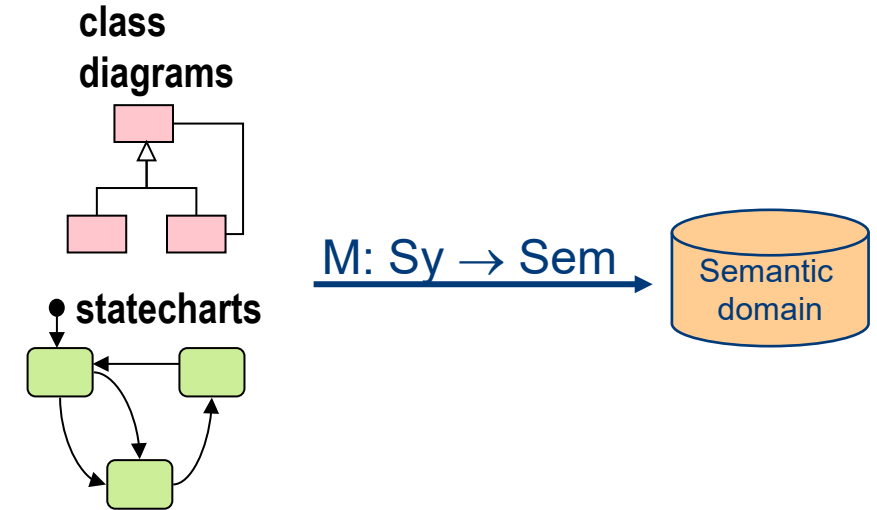
- Charakteristika:
 - Web-App für manuelle Zeitmessung bei Rennen aller Art
 - Konfiguration für verschiedene Renn-Settings, Teilnehmer-Verwaltung, Zeitmessung im Rennen, Live Ergebnisse,...



Nützlichkeit von Modellierung

Im Entwicklungsprozess werden

- Modelle erstellt
- Modelle diskutiert und weiterentwickelt
- Modelle verglichen
- Modelle analysiert
- Modelle zur Generierung oder Simulation eingesetzt



Das funktioniert am besten, wenn

- es eine präzise Definition gibt, was ein wohlgeformtes Modell ist (**Syntax**)
- es eine präzise Definition gibt, was ein Modell bedeutet (**Semantik**)
- es eine zugrunde liegende (mathematische) Theorie gibt, die Werkzeuge liefert, Modelle zu analysieren, zu transformieren etc.



ÜBUNGSaufGABE 1.1

Modell eines Computers

Modell eines Computers

Erstellen Sie ein Modell aus der Beschreibung eines „Computers“ (natürlich stark vereinfacht)

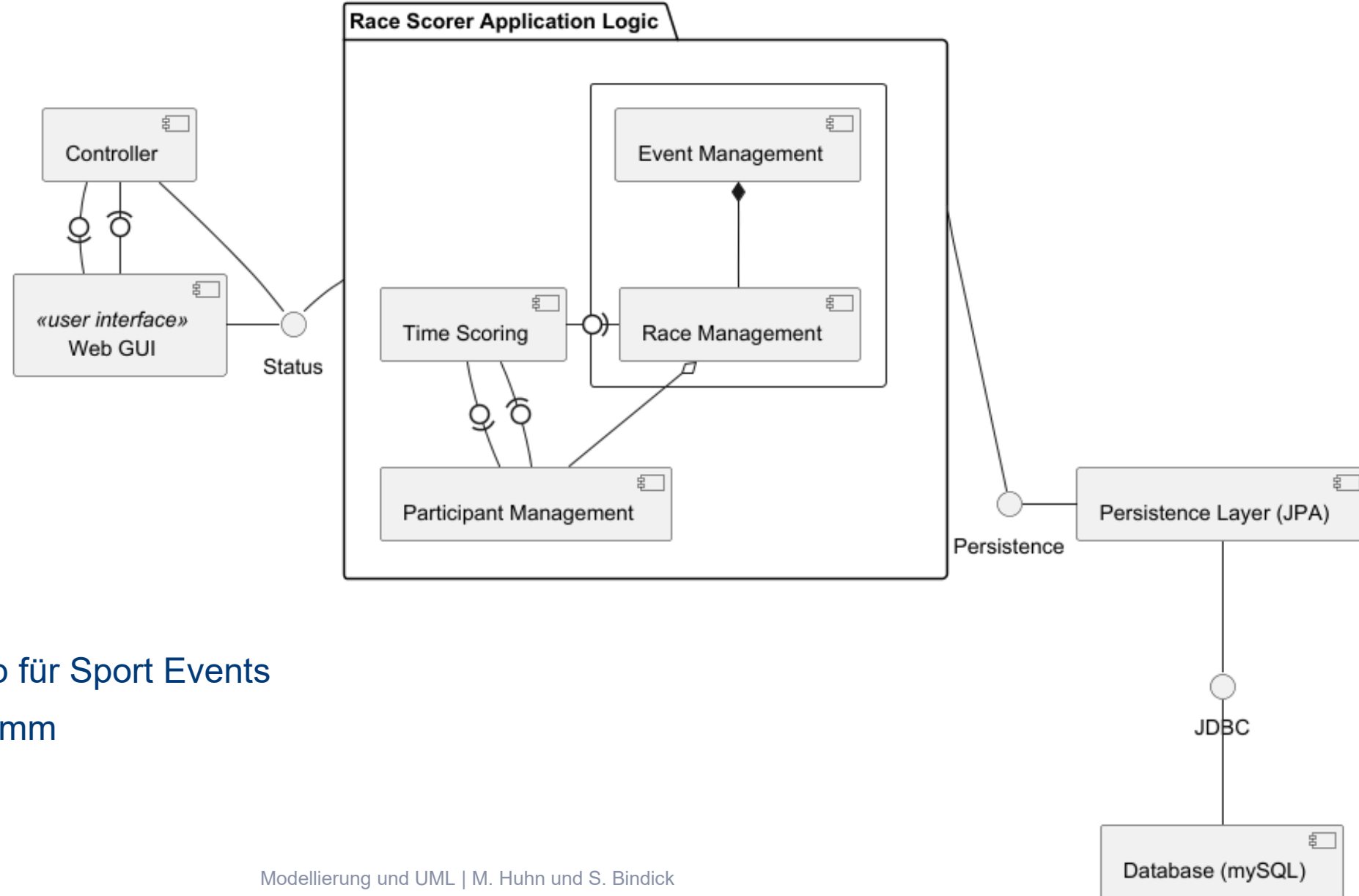
1. Ein Computer besteht aus mehreren Komponenten, darunter Gehäuse, Motherboard, CPU und Speicher. Der Speicher hat eine Kapazität, die CPU eine Taktfrequenz. An den Computer kann Peripherie angeschlossen werden, z.B. ein Display. Computer können mit anderen Computern verbunden sein.
2. Zu Beginn ist ein Computer ausgeschaltet. Betätigt man den Ein-Schalter oder drückt man eine Taste auf der Tastatur, wird der Computer hochgefahren. Ist das Betriebssystem fertig geladen, ist der Computer betriebsbereit. Wenn der Computer abstürzt, friert er ein und kann nur durch ziehen des Netzsteckers ausgeschaltet werden. Normalerweise schaltet man den Computer mit dem Ein-Schalter auch wieder aus.



ÜBUNGSaufGABE 1.2

Aussagen zu Komponentenmodell

Aufgabe



Time Scoring Web-App für Sport Events

- Komponentendiagramm

Aufgabe: Welche Aussagen zu diesem Komponenten-Diagramm sind falsch?

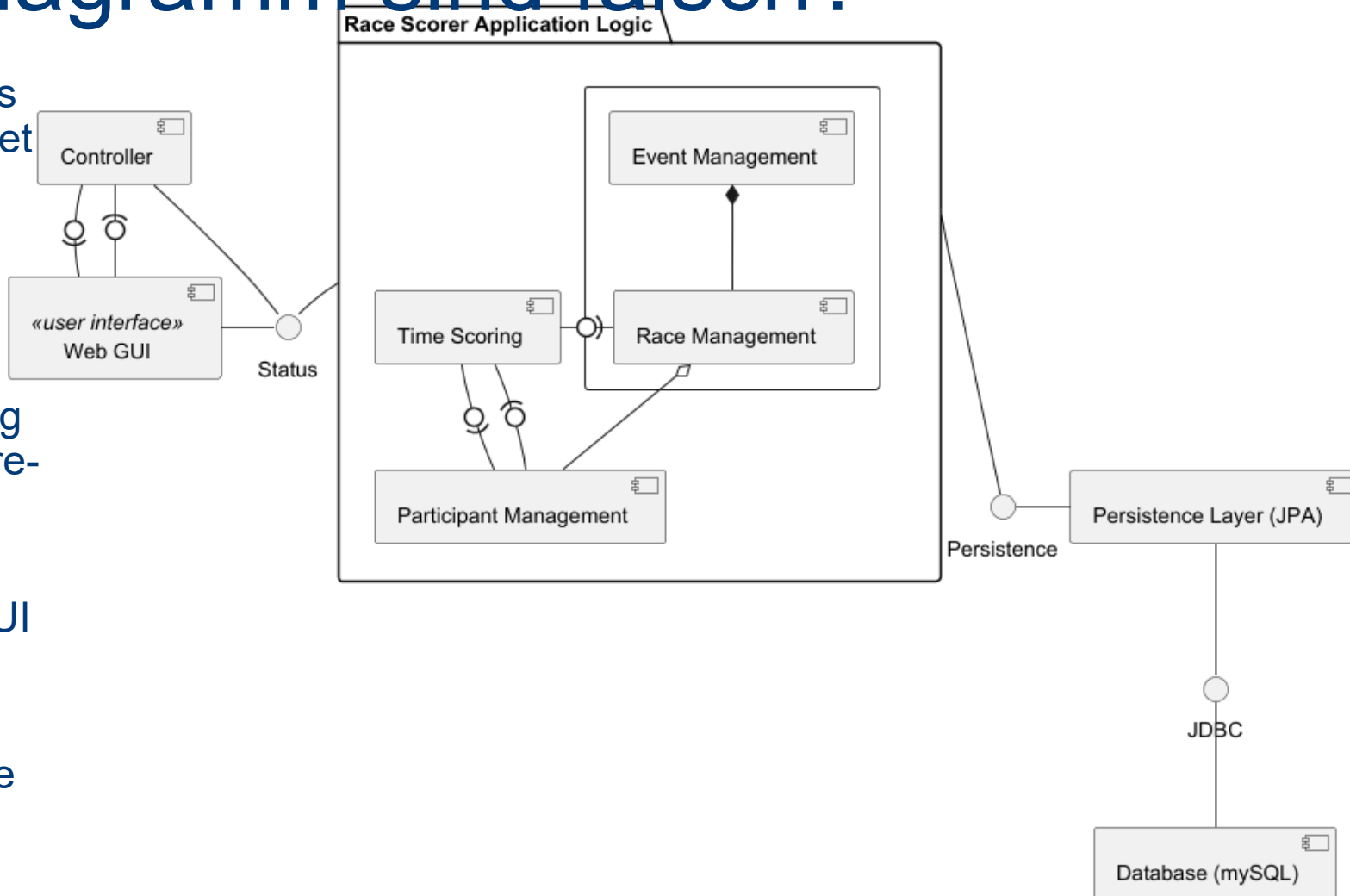
A Das Modell kann als ein präskriptives Software-Architekturmodell verwendet werden.

B Das Modell abstrahiert von der Klassenstruktur.

C Ziel des Modells ist die Unterstützung der Kommunikation über die Software-Architektur.

D Das Modell beschränkt die Kommunikation zwischen Frontend (Web GUI und Controller) und der Application Logic auf die Schnittstelle "Status".

E Das Original zu diesem Modell ist die Anforderungsspezifikation (Lastenheft).



ZUSAMMENFASSUNG

Zusammenfassung

Modellbasierte Softwareentwicklung

- nutzt Modelle als zentrales Artefakt in der Softwareentwicklung:

Ein **Modell** gehört zu einem **Original**, ist eine **Abstraktion** des Originals und hat einen auf das Original bezogenen **Einsatzzweck**.

Die **UML** ist Industriestandard bei der Modellierung von Softwaresystemen.

Verschiedene **Sichten der UML** dienen zur

- Analyse von Eigenschaften des Originals oder zur
- konstruktiven Generierung von Code oder Tests